A-C01 Cheat Sheet — Public Reference Edition

erified against AWS's official exam guide · docs.aws.amazon.com · checked 15 July 2026

Questions	Duration	Scored / Unscored	Pass score	Domains
65 total	130 minutes	50 scored / 15 unscored	720 (scaled 100–1000)	Data 28% · Model Dev 26% · Deploy 22% · Monitor 24%

Confirmed directly against AWS's current MLA-C01 exam guide: question counts, timing, scaled pass score, and domain weightings all match exactly.

Exam Strategy

Four-step triage: 1) Underline hard limits — latency, payload, duration, frequency, cost, compliance. 2) Identify the stage: prepare, train, deploy, or operate. 3) Eliminate services with a different core responsibility. 4) Pick the most-managed option that satisfies every constraint.

Pacing: First pass ~90 min, review ~40 min. Multiple-response questions need every correct choice — no partial credit. No guessing penalty, so never leave a blank. For ordering questions, think in pipeline-stage order (ingest → transform → store/serve).

Question formats you'll see: single-answer multiple choice · multiple response (select two/three) · ordering/sequencing (3–5 steps) · matching (technique ↔ use case).

Correction: AWS's current exam guide lists exactly these four question types. It does **not** currently list a "case study" format — if you see that claim elsewhere, it's likely carried over from an older guide version.

Classic distractors: Async = large/long single requests. Batch Transform = an entire offline dataset. Serverless = sporadic online traffic. A shadow variant never returns predictions to users. Don't confuse these.

1 · Data Preparation for Machine Learning — 28% of exam

Ingest & store data · transform & engineer features · ensure data integrity

Task 1.1 · Ingest & store data

Service	What it's for
`Amazon S3`	Data lake & artifact store. Partition by common filters; prefer Parquet/ORC for query cost + speed.
`Glue Crawler + Data Catalog`	Discovers schema/partitions and registers metadata for querying.
`AWS Glue ETL`	Serverless managed Spark. Best tool to convert bulk CSV/JSON → Parquet at scale for cost + query performance.
`Amazon Athena`	Serverless SQL over S3 only — cannot query Redshift or Snowflake directly.
`AWS Glue DataBrew`	Visual/no-code prep with reusable recipes. Requires every file in a source folder to be the same type — split mixed CSV/JSON/XLSX/Parquet into per-type folders first. Its masking transforms preserve statistical relationships (better than random-value overwrite via Batch/EMR). Doesn't output YAML.
`SageMaker Data Wrangler`	Built-in connectors query Redshift & Snowflake directly via SQL — no separate streaming/Glue hop needed. A flow can target a Feature Store as its destination.
`AWS Lake Formation`	Centralized data-lake access governance.
`SageMaker Ground Truth`	Managed human labeling.
`SageMaker Feature Store`	Online (low-latency) store for serving + offline store for training/batch.
`Amazon Macie`	Discovers/alerts on sensitive data in S3 — cannot mask or categorize it. Pair with DataBrew or Glue to remediate.
`Amazon Comprehend`	Detects/redacts PII (incl. in PDFs) via an async analysis job, writes results back to S3. Also does entity recognition & sentiment — not text-to-speech.

Amazon EMR — managed Hadoop/Spark clusters. Choose over Glue ETL when you need direct cluster control (custom libraries, long-running interactive Spark/Hive jobs, existing Hadoop ecosystem code) rather than a fully serverless ETL job.

Amazon Data Firehose — simple, serverless delivery of streaming records to S3/Redshift/OpenSearch, no infra to manage. It delivers data downstream; it is not the ingest buffer itself (that's Kinesis Data Streams).

Bias workflow, least overhead: EDA in Data Wrangler → run a Clarify bias report → review the report. That exact order shows up as a sequencing question.
Data Wrangler → Feature Store pipeline order: set a feature group as the flow's destination → run the Data Wrangler processing job → read features back from the offline Feature Store.

Storage services

Need	Use
High-throughput HPC/ML training (video, big data)	`FSx for Lustre`
Shared concurrent access, many EC2 instances (NFS)	`Amazon EFS`
Attached storage, single instance	`Amazon EBS`
On-prem ↔ AWS hybrid storage	`AWS Storage Gateway`
One-time/scheduled data migration	`AWS DataSync`
Cache frequently accessed data from on-prem/S3	`Amazon File Cache`
Extreme low-latency I/O for training/real-time feature retrieval, single-AZ	`S3 Express One Zone`
Cold storage for archived training data, logs, old model artifacts	`Amazon S3 Glacier`

Data formats

Format	Notes
`Parquet`	Columnar, compressed — best choice for large-scale ML data storage & query performance.
`CSV`	Simple, widely supported, inefficient at scale (no compression/indexing).
`Avro (RecordIO)`	Row-based, efficient serialization, supports schema evolution; less efficient than Parquet for analytics.
`XML`	Verbose, hierarchical; poor fit for big-data storage/processing.

Task 1.2 · Transform & engineer features

Symptom / goal	Technique
Right-skewed numeric column	Logarithm transformation
Date field → year / month / day	Feature splitting
Numeric → categorical buckets	Binning (quantile binning = equal-count bins)
Interactions between categorical/text vars	Cartesian product transformation
Text n-gram style analysis	Orthogonal sparse bigram (OSB)
Class imbalance – tabular (XGBoost)	Raise `scale_pos_weight` — never lower it — to penalize missing the minority class more
Class imbalance – images	Random oversampling of the minority class (duplicate/synthesize). Rotation/resize and dataset-size changes do not fix imbalance.
Mask sensitive but structurally-meaningful features	AWS Glue DataBrew built-in masking — preserves relationships, unlike random-value substitution

> **AWS Glue Data Quality** — rule-based checks for completeness, freshness, and schema conformance inside Glue. Complements DataBrew/Catalog; it is not a substitute for Clarify's bias metrics.

Data Wrangler exploratory visualizations

Goal	Chart
Correlation between 2 continuous variables, spot outliers	Scatter plot
Frequency/distribution of a single variable	Histogram
Feature importance (post-training, not exploratory)	Shapley values plot
Categorical counts over a period	Bar chart
Patterns across 2 dimensions (e.g., time × category)	Heatmap

Task 1.3 · Ensure data integrity & prep for modeling

- Split into train/val/test *before* fitting imputation, encoding, or scaling.
- Fit transforms on train only; persist and re-apply the same transform at inference.
- Time series: train on earlier data, validate on later data — never shuffle across time.
- Validate schema, missingness, ranges, duplicates, and label quality before training.
- Remove unnecessary identifiers; watch for target/future leakage.

`S3DataDistributionType` **on a training job:** `FullyReplicated` — the entire dataset is copied to every training instance. `ShardedByS3Key` — only a subset/shard is copied to each instance — use for large data or when each instance should see different data.

2 · ML Model Development — 26% of exam

Choose a modeling approach · train & refine · analyze performance

Task 2.1 · Choose a modeling approach

Built-in algorithm / JumpStart — fastest managed baseline or pre-trained starting point.
Script mode — bring existing code in a supported framework.
Custom container — for an unsupported framework (e.g. JAX) or special dependencies: build a Docker image, push to ECR, use it in SageMaker. This is the only way to make a custom framework immediately available to all notebook users — per-notebook `requirements.txt` installs aren't immediate or consistent.

Task	Built-in algorithm
Binary classification, minimize false positives	LightGBM / XGBoost / CatBoost — optimize for precision
Binary classification, minimize false negatives	Same algorithms — optimize for recall
Simple supervised classification on labeled data	k-NN
Anomaly detection (unsupervised)	Random Cut Forest (RCF) — not for classification
Clustering (unsupervised)	K-means
Time-series forecasting	DeepAR

NLP / text algorithms

Algorithm	Notes
Latent Dirichlet Allocation (LDA)	Unsupervised topic modeling — discover unknown/unlabeled topics in a text corpus.
BlazingText (Text Classification mode)	Supervised — requires labeled data.
Text Classification – TensorFlow	Supervised, predefined classes.
Sequence-to-Sequence	Translation / summarization.

Ensemble methods

Method	How it works
Bagging (e.g. Random Forest)	Trains multiple models in parallel on random data subsets and averages predictions — reduces variance.
Boosting (e.g. XGBoost, LightGBM)	Trains models sequentially, each correcting the previous one's errors — reduces bias.
Stacking	Combines different model types (e.g. SVM, Random Forest, k-NN) using a meta-model that learns how to weight each one for the final prediction — the right choice when base models are architecturally diverse.

Overfitting (high variance, low bias) — remedy: L1/L2 regularization, dropout, fewer features, early stopping. **Underfitting** (high bias, low variance) — remedy: more model complexity, more features, less regularization.

AI services by use case

Task	Service
Fraud detection (custom models)	`SageMaker AI`
Fraud detection (standard patterns, no ML expertise needed)	`Amazon Fraud Detector`
Recommendation engine (standard, managed)	`Amazon Personalize`
Sentiment/insights from text	`Amazon Comprehend`
Medical text extraction	`Amazon Comprehend Medical`
Store/query health data in FHIR format	`AWS HealthLake`
Chatbot / conversational interface	`Amazon Lex`
Image/video analysis	`Amazon Rekognition`
Text extraction from documents/forms (OCR)	`Amazon Textract`
Speech-to-text	`Amazon Transcribe`
Text-to-speech	`Amazon Polly`
Language translation	`Amazon Translate`
Enterprise document search	`Amazon Kendra`
Predictive maintenance from sensor data	`Amazon Lookout for Equipment`
Anomaly detection in business/operational metrics	`Amazon Lookout for Metrics`
Automated visual defect inspection	`Amazon Lookout for Vision`
Human review of low-confidence *live* predictions	`Amazon A2I`
On-demand crowdsourced labeling workforce	`Amazon Mechanical Turk`

A2I reviews inference-time outputs; Ground Truth labels training data; Mechanical Turk is one of the workforce options Ground Truth can route labeling jobs to (vs. a private or vendor workforce).

Task 2.2 · Train & refine models

Option	When
GPU	Accelerates deep-learning tensor work.
Spot Training	Cheapest for one-off/occasional jobs that tolerate interruption + checkpointing.
SageMaker Savings Plan (1–3 yr)	Best when usage is steady and predictable over many weeks (e.g. 35 hrs/wk × 55 wks) — beats Spot for long-term consistent load. EC2 Instance Savings Plan is the EC2-only analog.
SageMaker Training Plan	Reserves GPU (and Trainium/Inferentia) capacity only — supported instance types are P-, Trn-, and UltraServer families; it does not support compute-optimized (C-type) instances.
Heterogeneous clusters	Mixed instance types/groups in one training job, for resource-utilization optimization — not for steady-state cost savings.
Managed warm pools	Reuses compatible infrastructure between sequential jobs, cutting startup latency. Cheaper than HyperPod for bursty/idle-some-days workloads. Set `keep_alive_period_in_seconds`.

SageMaker Experiments — tracks params, metrics, data/artifact lineage across trials. **Model Registry** — versioned model packages + approval workflow. **Model Card** — documents intended use, limits, evaluation, and risk context.

HPO strategies

- **Bayesian:** uses prior trial results to pick promising next trials.
- **Hyperband:** early-stops weak trials — great for expensive deep learning.
- **Random:** simple, broad baseline.
- **SageMaker automatic model tuning:** tunes hyperparameters while keeping the same model/algorithm — least effort. Autopilot (even with ProblemType set) may swap the underlying algorithm, so it's the wrong pick when the model type must stay fixed.

> **SageMaker Debugger** (still an active, current service): real-time detection of training problems — vanishing/exploding gradients, overfitting, poor weight initialization — via hooks and built-in rules, with CloudWatch/Lambda-triggered actions such as auto-stopping a diverging job. Only its older "framework profiling" feature is deprecated for newer TensorFlow (2.11+) and PyTorch (2.0+); built-in profiler rules must now be added explicitly rather than being on by default. Keep it separate from Model Monitor, which watches data/model quality *after* deployment, not during training.

> **Managed MLflow on SageMaker** — a fully managed MLflow Tracking Server (or "MLflow Apps" in SageMaker Unified Studio) for experiment tracking and a Model Registry that syncs into the native SageMaker Model Registry. Use it when a team already standardizes on the open-source MLflow API instead of, or alongside, native SageMaker Experiments.

Task 2.3 · Analyze model performance

- **Recall:** catch positives; false negatives are costly (e.g. "catch all fraud").
- **Precision:** alerts must be correct; false positives are costly.

- **False Positive Rate (FPR):** proportion of genuine cases wrongly flagged — minimize when false alarms carry real cost.
- **F1:** balances precision and recall. **PR-AUC:** especially useful with rare positives (imbalanced data).
- **AUC-ROC:** discrimination ability across all classification thresholds — use when you need a single, threshold-independent score.
- **RMSE:** penalizes large numeric errors more than MAE (regression only, not classification).

Pre-training bias metrics (SageMaker Clarify)

Metric	What it measures
Class Imbalance (CI)	Imbalance between group sizes (e.g., under-representation of one gender).
Difference in Proportions of Labels (DPL)	Gap in positive-outcome rate between demographic groups.
Total Variation Distance (TVD)	Max divergence in outcome distributions between groups.
Conditional Demographic Disparity (CDD)	Prediction differences across demographics *after controlling for* legitimate differentiating factors (e.g., geography) — use this instead of a raw demographic-parity comparison whenever a legitimate confounding variable is in play.
KL Divergence	General distribution comparison — not the standard bias-specific metric on the exam.

Model complexity vs. explainability

Simple models (decision trees, logistic regression): high explainability — use when transparency/regulation matters (e.g., loan approvals). **Complex models** (deep nets, ensembles): higher accuracy, low explainability.

Amazon Bedrock & GenAI

If the scenario says…	Choose	Why
Answer from enterprise docs, no custom RAG plumbing	Bedrock Knowledge Bases	Managed ingest, chunk, embed, vector store, retrieve.
Decide & perform multi-step API actions	Bedrock Agents	Action groups orchestrate tool/API use.
Block unsafe content/PII; constrain behavior	Bedrock Guardrails	Safety/content controls — not a substitute for grounding/RAG.
Semantic document matching	Embeddings + vector search	Retrieve semantically similar chunks.
Current facts, no retraining	RAG	Retrieve trusted context at inference time.
NL query over many documents, no managed KB service specified	Amazon Kendra Retrieve API + pass chunks to an FM	Kendra does semantic/contextual search; the Bedrock API alone can't search docs without an index.
Teach a model company jargon/abbreviations cheaply	Fine-tuning	Cheaper & faster than continued pre-training or a new FM. A Knowledge Base is for fresh facts, not terminology.
Bias analysis + SHAP (offline)	SageMaker Clarify processing job	Pre/post-training bias + explainability; batch, not real-time.
Bias analysis + SHAP (real-time, per-prediction)	Endpoint config with Clarify ExplainerConfig	Returns prediction + SHAP attribution inline.
Text-to-speech with pauses between paragraphs	Amazon Polly + SSML break tags	Lexicons only customize pronunciation, not timing.
Embedded generative assistant inside an AWS console or business app	Amazon Q	A pre-built generative assistant layered on top of Bedrock/FMs; not the underlying platform itself.

Bedrock inference parameters (Anthropic Claude on Bedrock)

Goal	Adjust
More consistent, on-brand, less random output	Lower temperature, top_p, top_k — raising any of these increases randomness, the wrong direction for consistency
Mark precise content endpoints to keep output structured	stop_sequences
Force concise/brief output	max_tokens_to_sample — direct length control

RAG quality checklist: synchronize source changes · tune chunk size/overlap · use trusted, access-controlled source data · evaluate retrieval relevance and grounding · test quality, latency, safety, cost on representative tasks.

Responsible release: define intended/unacceptable use · assess bias, review attributions · use guardrails and output validation · document limits in a Model Card · monitor business and safety signals post-launch.

Traps: Knowledge Bases ≠ Agents (retrieve vs. act). Guardrails ≠ a RAG substitute. Prompt-only control is insufficient — validate outputs and enforce access controls too.

3 · Deployment & Orchestration — 22% of exam

Select deployment infra · script infra · CI/CD orchestration

Task 3.1 · Select deployment infrastructure

Need	Choose	Remember
Low-latency synchronous requests	Real-time endpoint	Provisioned capacity; autoscale it; cannot scale to 0.
Intermittent/sporadic traffic, cold start OK	Serverless inference	Pay per use, scales to 0. Cap with MaxConcurrency. Add provisioned concurrency for low latency on a predictable schedule without managing infra. Does **not** support GPU acceleration.
Up to 1 GB payload / up to 60 min; result later	Asynchronous inference	Queues requests, writes results to S3. InitialInstanceCount min is 1 at creation.
Offline scoring of a whole dataset	Batch Transform	No always-on endpoint; not ideal for tiny <1 min jobs — compare serverless for those.
Many tenant models, shared fleet, cold start OK per model	Multi-model endpoint (MME)	Models load on demand from S3, share instance memory/GPU; scaling policy is per-endpoint/instance, not per model.
Multiple models, each needs its own scaling + no cold start, on accelerated instances	Inference Components (IC)	Per-model compute (incl. GPU) + independent auto scaling on one endpoint; set min copies ≥1 to avoid cold start.
Preprocess → model → postprocess in one request	Inference pipeline	Serial multi-container path, one endpoint.
Repeated short jobs back-to-back, same config, cost-sensitive	Managed warm pools	Set keep_alive_period_in_seconds. Cheaper than HyperPod for bursty/idle-some-days workloads.

> **Deprecation watch-outs:** Amazon Elastic Inference (EI) was fully discontinued (retired April 2023) — today's equivalents are Inferentia-based instances, right-sizing, or multi-model/Inference Components endpoints. SageMaker Edge Manager was discontinued on 26 April 2024 and can no longer be used — for edge deployment, the current guidance is AWS IoT Greengrass V2 deploying SageMaker Neo-compiled models (or an ONNX runtime).

Deployment / rollout strategies

Strategy	Behavior	Use when
Production variants (weighted)	Both variants return live predictions to real users, split by weight.	A/B test needing real customer feedback; raise the new variant's weight gradually.
Shadow variant	Candidate gets a copy of traffic but its response is never shown to users.	Validate a new model silently in prod — not for collecting user feedback.
Blue/green – all-at-once	Instant full cutover.	Fast release, rollback = revert environment.
Blue/green – canary	Small fixed % first, then jump to 100%.	Quick validation, not a gradual daily ramp.
Blue/green – linear	Traffic shifts in equal steps over set intervals (e.g. +10%/day), with rollback.	Scenario cue: "increase exposure by X% each day, must be able to roll back."
Rolling	Replace instances one/batch at a time.	Gradual infra replacement; weaker rollback story than blue/green.

Task 3.2 · Script infrastructure

Kinesis Data Streams → Managed Service for Apache Flink → SageMaker: real-time ingest → stream analysis → fraud/anomaly scoring. Highly available real-time pipeline. Bedrock cannot read directly from a Kinesis stream; Autopilot/Data Wrangler are not real-time ingestion tools. (Amazon Data Firehose is the simpler alternative when you just need delivery to S3/Redshift/OpenSearch with no custom transformation on the stream.)

AWS Glue ETL, serverless, at scale: store raw in S3 → Glue ETL job → output as Parquet for cost + query performance. Ground Truth/Model Monitor are not ETL tools; Amazon EFS isn't directly readable by Glue.

Managed warm pools for thousands of short jobs: `keep_alive_period_in_seconds` — cheapest for short/intermittent/sequential jobs. HyperPod is a persistent cluster that keeps costing even when idle, wrong for intermittent work (see Appendix). AWS Batch + Spot risks interruption delays between jobs.

Task 3.3 · CI/CD orchestration

SageMaker Pipeline steps: `ProcessingStep` transforms/evaluates. `TrainingStep` / `TuningStep` train or optimize. `RegisterModel` / `ModelStep` packages/registers — can be set to `PendingManualApproval` so a reviewer must approve before deployment, creating an auditable governance trail. `ConditionStep` deploys only when quality passes. `FailStep` stops deliberately with a reason. Step cache reuses unchanged work; Parameters reuse the pipeline across environments.

Automation & orchestration services

Service	Use
Amazon EventBridge	Scheduled/event trigger.
CodePipeline + CodeBuild	Source, build, test, promote (software/CI-CD, not ML-specific governance).
AWS Step Functions	General stateful workflow with retries/integrations; supports SageMaker but lacks native ML lineage/registry features without custom code.
Amazon MWAA (Managed Airflow)	DAG-based orchestration for complex, code-first workflows with dependencies; requires managing/paying for the environment, no native SageMaker lineage tracking.
IaC (CloudFormation / AWS CDK)	Consistent environment configuration.
AWS CodeArtifact / CodeDeploy	Private package/artifact storage and deployment automation inside a CI/CD pipeline.
Amazon SNS / SQS	SNS = pub/sub notifications (classic pattern: CloudWatch alarm → SNS email). SQS = decoupling queue between pipeline stages.
AWS X-Ray	Distributed tracing across microservices/Lambda calls — useful for diagnosing latency across a multi-service inference pipeline.
AWS Serverless Application Repository	Catalog of prebuilt, deployable serverless application templates (e.g., SAM-based Lambda apps) — speeds up wiring common serverless patterns instead of building from scratch.

4 · Monitoring, Maintenance & Security — 24% of exam

Monitor inference · optimize infra & cost · secure resources

Task 4.1 · Monitor model inference

Service	Role
`SageMaker Model Monitor`	Tracks data quality/drift, and with labels, model quality. Runs on a scheduled batch cadence (e.g. hourly/daily) against a baseline — not real-time.
`SageMaker Clarify`	Bias/explainability monitoring — batch, or inline via ExplainerConfig for real-time.
`Data Capture`	Retains request/response samples for analysis.
`Amazon CloudWatch`	Metrics, logs, alarms. Can trigger e.g. an SNS email on a threshold breach — Lambda logs metrics to CloudWatch, a CloudWatch alarm sends the email. CloudFront/CloudTrail can't do this. Native SageMaker endpoint metrics (e.g. Invocations) support CloudWatch anomaly-detection alarms for unusual traffic without any extra infrastructure.
`CloudWatch Logs Insights`	Purpose-built query language for real-time log analysis; its ML-powered pattern command auto-detects and summarizes patterns across large volumes of log lines.
`AWS CloudTrail`	API/audit history — who changed what.
`AWS Config`	Tracks and audits resource *configuration* changes over time — complements CloudTrail (who did what) by recording what a resource's configuration looked like at any point in time.
`AWS Chatbot`	Relays CloudWatch alarms/notifications into Slack or Chime channels for team visibility.

Set a representative baseline before monitoring, and track business KPIs alongside ML metrics.

Scaling policies

- **Target tracking:** auto add/remove instance count to hold a metric (predefined or custom) at a target — correct for "handle sporadic/unpredictable traffic cost-effectively."
- **Step scaling:** add/remove instances by alarm steps; cannot change instance type.
- **Scheduled scaling:** only for predictable time-based patterns, not sporadic traffic — and still means you manage the EC2 infra yourself.
- Neither policy type can swap instance types — only instance count.

Task 4.2 · Optimize infrastructure & cost

SageMaker Studio built-in cost tag: activate the `sagemaker:` user-level cost allocation tag — appears in Cost Explorer in ~24–48 hrs. Least overhead vs. a custom Lambda tagging function.

AWS Budgets, fixed method: fixed method + user-defined cost allocation tags (dept/project) + alert on forecasted spend = anticipate overage before it happens. Auto-adjusting resets the threshold based on past spend, so it won't reliably flag "about to exceed a set budget."

Right-size & autoscale: stop idle resources; use Spot only when interruption-tolerant. **AWS Compute Optimizer** analyzes utilization metrics and recommends right-sized SageMaker/EC2 instance types; **AWS Trusted Advisor** adds broader account-level cost, security, and performance checks.

Cost allocation tagging (correct order)

1. Create user-defined tags (key-value pairs) for the ML resources.
2. Attach the tags to the ML resources in the AWS Management Console.
3. Enable the cost allocation tags in the AWS Billing console.
4. Configure tag-based Cost & Usage Reports for detailed analysis.

AWS Billing and Cost Management is the umbrella console housing invoices, cost allocation tags, Budgets, and Cost Explorer.

Task 4.3 · Secure AWS resources

IAM role & condition keys: task-specific least privilege; no embedded keys. IAM condition keys (e.g. `sagemaker:RootAccess`) can preventatively block creation of noncompliant resources — better than reactive EventBridge/Lambda cleanup.

KMS: encrypts data at rest only. Switching a bucket to SSE-KMS breaks training jobs with `AccessDenied` unless you also add `kms:Encrypt` / `kms:Decrypt` to the training job's execution role (not the user's IAM policy). If a job is already correctly permissioned but still times out, suspect KMS request throttling — fix with an S3 Bucket Key, which cuts the volume of calls to KMS.

AWS Secrets Manager: stores and rotates credentials, API keys, and other secret values — distinct from KMS, which manages the encryption keys themselves.

Encrypt data in transit between training nodes: a distinct setting from a KMS key on the job request — enable inter-node encryption explicitly for distributed training.

Private network, S3 access, no internet: run in private subnets + an S3 interface (VPC) endpoint. A NAT gateway still allows

outbound internet, so it fails a strict "no internet" requirement. This is the general VPC endpoint / PrivateLink pattern: a private service path to any supported AWS service from inside a VPC, no internet hop.

Container network isolation: `enable_network_isolation=True` on a training/processing job blocks that job container's outbound network access entirely — separate from VPC subnet placement.

SageMaker Studio, fully private: "VPC only" domain mode + interface VPC endpoints = no internet path at all.

Opt out of library telemetry: `OPT_OUT_TRACKING=1` on `CreateTrainingJob` stops SageMaker from collecting metadata about AWS-provided library usage (0 = allow, the opposite).

Cross-account S3 sharing (console + programmatic access): create an IAM role in the source account that the partner account can assume. A bucket policy alone covers programmatic access but not console browsing; ACLs are discouraged and don't help either.

One-Minute Recall

Service pairs

- Feature Store = shared features · Experiments = run lineage · Registry = approval/version
- DataBrew = visual transforms (same-type folders only) · Glue = code-based ETL · Athena = SQL on S3 · Lake Formation = governance
- Clarify = bias/SHAP · Model Monitor = live data/model quality · CloudWatch = telemetry/alarms · CloudTrail = audit · Config = configuration history
- Knowledge Base = retrieval · Agent = action · Guardrail = safety
- Macie detects sensitive data only; DataBrew masks it
- HyperPod = long-running, resilient, large-scale training · Managed MLflow = open-source-compatible experiment tracking integrated with SageMaker · A2I = human review of live, low-confidence inference (vs. Ground Truth = training-time labeling)

Scenario cue → answer

- Sporadic online traffic → serverless / target tracking
- Large/slow online request → async
- Bulk offline prediction → Batch Transform
- Per-model scaling, no cold start → Inference Components
- Catch all fraud → recall · alerts must be correct → precision
- Past trials guide next → Bayesian HPO · stop weak trials early → Hyperband
- Quality threshold gates release → ConditionStep
- New model sees live traffic invisibly → shadow variant
- Daily % ramp with rollback → blue/green linear shifting
- Predictable steady weekly hours, many weeks → Savings Plan
- Short jobs back-to-back, gaps some days → managed warm pools

Final checklist

- Know the four domain weights and where the team is weakest.
- Differentiate every service pair above.
- Match metric to the business cost of an error.
- Recognize endpoint/inference constraints, including Inference Components & warm pools.
- Explain a gated pipeline (ConditionStep/FailStep).
- Describe the drift → baseline → alarm → remediation loop.
- Apply IAM/KMS/VPC least-privilege, including execution-role KMS grants and S3 interface endpoints.
- Know Bedrock inference params: lower temp/top_p/top_k for consistency, stop_sequences/max_tokens for structure/length.

Do not memorize question dumps. Use official AWS material and reputable explanatory practice. When a third-party source conflicts with AWS documentation, trust the docs.

Appendix · Newer & Advanced Items

Watch-list items for context — useful background, not heavily tested unless referenced directly in a current AWS exam-guide revision.

SageMaker HyperPod

Purpose-built, persistent, resilient cluster infrastructure for distributed training/fine-tuning of large or foundation models across hundreds to thousands of accelerators, orchestrated via Slurm or Amazon EKS.

- Automatic faulty-node detection, replacement, and auto-resume from checkpoint — designed to run for weeks/months without manual intervention.
- Managed tiered checkpointing (CPU memory + periodic S3 persistence) for fast recovery after a failure.
- Task governance: queues and prioritizes compute across training, fine-tuning, experimentation, and inference tasks.
- Gang scheduling: holds a distributed job until all its pods are ready, avoiding partial-job waste and deadlocks.
- Elastic training: absorbs idle accelerators automatically, then yields them back when higher-priority work (e.g. inference) needs the capacity.

When to choose it vs. alternatives: HyperPod = long-running (weeks/months), large-scale, distributed FM training/fine-tuning. Standard SageMaker Training Jobs = ephemeral, on-demand, single runs or HPO sweeps. Managed warm pools = many short, back-to-back, similarly-configured jobs. Training Plans = reserved GPU/Trainium/UltraServer capacity for a planned window — it doesn't provide HyperPod's own resiliency/checkpointing layer.

Amazon Bedrock AgentCore

A separate, broader platform for building, deploying, and operating AI agents in production — covering session isolation and runtime, identity/permissions (AgentCore Identity), tool access via the Model Context Protocol (AgentCore Gateway), and automated quality evaluation (AgentCore Evaluations) — independent of which agent framework or foundation model is used.

How it differs from Bedrock Agents: Bedrock Agents is the original, simpler feature — a single Bedrock-hosted agent whose action groups orchestrate API/tool calls. AgentCore is the newer, framework-agnostic production platform (works with LangChain, CrewAI, Strands, and others, with any model) for running agents securely at scale with observability built in. It reached general availability only recently, so treat it as "know it exists and how it differs from Bedrock Agents" rather than a heavily-tested topic.

SageMaker Unified Studio & the lakehouse architecture

A single governed workspace, built on Amazon DataZone, that unifies EMR, Glue, Athena, Redshift, MWAA, Bedrock, and SageMaker AI. It's built on an open lakehouse architecture (Apache Iceberg-compatible) that lets training jobs query or stream S3 and Redshift data as one logical copy, avoiding a separate ETL/copy step (a "zero-ETL" pattern). If a scenario calls for "one governed environment for data and ML/GenAI work across teams" or "avoid duplicating data between S3 and Redshift before training," this is the concept being tested — even though the exam guide still lists the underlying pieces (Redshift, Glue, Athena, SageMaker) separately rather than naming "Unified Studio" itself.

Sources: AWS MLA-C01 Exam Guide and in-scope/out-of-scope services pages (docs.aws.amazon.com/aws-certification), cross-checked 15 July 2026. Service capability descriptions reflect established AWS documentation as of the same date. When a specific answer choice conflicts with current AWS docs on exam day, trust the docs — AWS updates services faster than any static reference can track.